# claude-windows / 166d7059-d8ac-4fdd-a8c0-072cdaa7eccd

## Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\166d7059-d8ac-4fdd-a8c0-072cdaa7eccd\subagents\workflows\wf_66c34d9d-057\agent-a81c8b229b3f7f960.jsonl`
- SHA-256: `0c7672c7d3c0ead263cbed92fa20122246ba530ad1a56162e58a43cada508a0c`
- Source modified: `2026-06-18T05:37:53+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_66c34d9d-057`
- session_id: `166d7059-d8ac-4fdd-a8c0-072cdaa7eccd`

## Transcript

### 1. user (2026-06-18T05:35:27.896Z)

```
You are the adversarial synthesis judge.
WORKTREE (cd here; code is origin/master f5f339a): C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/verify-n13
Run python as:  cd "C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/verify-n13" && PYTHONPATH=. python <your_script>
GOLDEN/manifest/features dir (ABSOLUTE ? pass this, the worktree output/ is empty): C:/Users/avidu/Projects/badminton-highlight-indexer/output

REPRODUCE RECIPE (use the public API; write your own short script to scratch/lens_<name>.py):
  import os
  from dataclasses import replace
  from backend.eval import rally_seg_eval as rse, eval_stats
  from backend.eval.windowing import SERVED
  from backend.eval.tolerance_metrics import over_seg_guard
  OUT = "C:/Users/avidu/Projects/badminton-highlight-indexer/output"
  golden = [g for g in rse.load_golden(os.path.join(OUT,"human_lovo_manifest.json"), OUT)
            if g["gts"] and os.path.exists(g["trajectory"])]   # expect 13
  baseline  = SERVED                                          # all windowing knobs OFF
  cap12     = replace(SERVED, max_window_duration=12.0)
  cap6      = replace(SERVED, max_window_duration=6.0)
  milestone = replace(SERVED, max_window_duration=6.0, inactivity_end=1.5,
                      inactivity_rally_thresh=0.20, inactivity_min_density=0.30)
  rows = lambda p: {r["name"]: r for r in rse.evaluate(golden, p)["rows"]}  # each row: tol_f1,
      # f1, merge_rate, split_rate, merge_count, split_count, num_pred, num_gt, name, dend_abs_median
  # paired sign-test on a metric: eval_stats.paired_delta_ci([base[n][k] for n in names],[cand[n][k] for n in names])
  #   returns {mean_delta, ci_low, ci_high, ci_excludes_zero, sign_test:{wins,losses,ties,p_value}}
  # over-seg guard: over_seg_guard(list(base_rows.values()), list(cand_rows.values()))
  #   returns {passes, strict_passes, base_net, cand_net, net_delta, no_merge_rate_regress, merge_rate_regress:[...]}
  # NOTE: rse.evaluate may print "R5 blob-fallback: forced ..." lines for the blob clip ? that is the
  #   R5 logger and is HARMLESS/irrelevant here (the milestone does NOT enable blob_fallback). Ignore it.

THE CLAIM TO FALSIFY: "The R1 milestone (W1 cap=6 + R4 inactivity-end=1.5/rally_thresh=0.20/min_density=0.30, no W2, no R5) still clears the net-over-seg promotion bar at n=13 ? a SIGNIFICANT
R2 tolF1 win over baseline AND no honest over-seg regression ? so config-flip PR #204 still stands."

MY (the orchestrator's) measured n=13 numbers, for you to INDEPENDENTLY confirm or REFUTE ? do
```

NOT
trust them, recompute:
  - baseline->milestone tolF1: ?+0.2222 [+0.1522,+0.2859], sign 12/0/1, p=0.0005
  - cap12->cap6 tolF1:         ?+0.1099 [+0.0703,+0.1526], sign 12/0/1, p=0.0005
  - cap6->+R4 tolF1:           ?+0.0397 [+0.0145,+0.0651], sign 9/1/3,  p=0.0215   (ONE loss ?
find it)
  - mean merge_rate baseline->milestone: 0.651 -> 0.161; over_seg_guard NET PASS, no merge_rate
regression
  - new clips: GX010137 tolF1 0.448->0.685, GX010141 0.333->0.431 (both improve)


Here are 5 independent verification
lenses (each ran real code at n=13):


```
[
  {
    "lens": "independent reproduction (own script, public API, n=13 golden)",
    "verdict": "confirms",
    "key_numbers": "n=13 (no drop). Mean tolF1: baseline 0.1372 -> cap12 0.2099 -> cap6 0.3198
-> milestone 0.3594. baseline->milestone tolF1 ?+0.2222 [+0.1558,+0.2894], sign 12/0/1,
p=0.0005, STANDS. cap12->cap6 ?+0.1099 [+0.0700,+0.1522], sign 12/0/1, p=0.0005, STANDS.
cap6->+R4 ?+0.0397 [+0.0155,+0.0652], sign 9/1/3, p=0.0215, STANDS. The one cap6->+R4 loss =
GX010137 (0.7273->0.6849, d=-0.0423). mean merge_rate baseline->milestone 0.6513->0.1610.
over_seg_guard PASS: base_net 1.3026 -> cand_net 0.4289, net_delta -0.8737,
no_merge_rate_regress=True (empty regress list). New clips GX010137 0.4478->0.6849, GX010141
0.3333->0.4314 (both improve).",
    "discrepancies": [
      "CI bounds differ only by bootstrap jitter (3rd-4th decimal), not material:
baseline->milestone CI mine [+0.1558,+0.2894] vs reported [+0.1522,+0.2859] (~0.003-0.004
wider); cap12->cap6 mine [+0.0700,+0.1522] vs [+0.0703,+0.1526] (~0.0003); cap6->+R4 mine
[+0.0155,+0.0652] vs [+0.0145,+0.0651] (~0.001). All CIs still exclude 0 on both sides; point
estimates, sign-tests (W/L/T) and p-values match exactly. No discrepancy in verdict.",
      "merge_rate reported as 0.651->0.161; my exact values 0.6513->0.1610 (rounds
identically)."
    ],
    "milestone_still_stands": true,
    "caveats": [
      "CI numerical differences are expected: paired_delta_ci uses bootstrap (n_boot=2000,
seed=0) so identical inputs give identical CIs; the small bound differences vs the orchestrator
imply a tiny resampling/version difference, but are immaterial (all CIs robustly exclude 0).",
      "A transition 'stands' was judged by CI-excludes-0 AND sign-test p<0.05; all three pass.
The cap6->+R4 step is the weakest (p=0.0215, one loss GX010137) but still clears the bar.",
      "over_seg_guard NET passes (net_delta -0.8737) but strict_passes=False (per-video count
rule blocks) ? this is the documented R1 design: the NET rate-based guard is the promotion gate,
not the strict count rule.",
      "Verified pure-logic eval path only; this is offline windowing scoring over persisted
trajectories, no GPU/pipeline run needed. 3 clips are ties under cap6->+R4 (mahadevpura_1 0.0,
mahadevpura_2, testlarge_short)."
    ]
  },
  {
    "lens": "leave-one-video-out robustness (adversarial), baseline->milestone R2 tolF1 at
n=13",
    "verdict": "confirms",
    "key_numbers": "Full n=13 baseline->milestone tolF1: D+0.2222 [0.1522,0.2859], sign 12/0/1,
p=0.0005 (REPRODUCED exactly). All 13 LOO drops (n=12 each) survive CI-excludes-0 AND sign
p<0.05. Worst-case drop = GX010128: D+0.2019 [0.1474,0.2586], sign 11/0/1, p=0.0010 (still
strongly significant). Best/strongest drop = mahadevpura_1 (the blob): D+0.2407 [0.1775,0.3015],
sign 12/0/0, p=0.0005 -- dropping the blob STRENGTHENS the verdict because it is the lone tie
(0.000->0.000) in full-n. LOO mean-delta range across all drops: [0.2019, 0.2407]; all CI-lows
>= 0.1436. Special-clip drops all survive: drop GX010137 D+0.2210 p=0.0010; drop GX010141
D+0.2325 p=0.0010; drop mahadevpura_1 D+0.2407 p=0.0005. CI robust across bootstrap seeds
0/1/2/7 for the worst drop. Supporting full-n (all reproduced): cap12->cap6 D+0.1099
[0.0703,0.1526] p=0.0005; cap6->+R4 D+0.0397 [0.0145,0.0651] sign 9/1/3 p=0.0215 with the ONE
loss = GX010137 (0.7273->0.6849, d=-0.0423); mean merge_rate 0.651->0.161; over_seg_guard NET
```

```
PASS (base_net 1.3026, cand_net 0.4289, net_delta -0.8737, no_merge_rate_regress True,
strict_passes False); new clips GX010137 0.4478->0.6849 and GX010141 0.3333->0.4314 both
improve.",
    "discrepancies": [],
    "milestone_still_stands": true,
    "caveats": [
      "Win is NOT driven by any single clip: removing the single biggest per-video gainer
(GX010128, full-n d=+0.4662) still leaves D+0.2019, p=0.0010 -- 11/13 of the remaining videos
individually improve in every fold.",
      "The full-n 'tie' (sign 12/0/1) is mahadevpura_1 (the blob clip), which scores tol_f1
0.000 under BOTH baseline and milestone -- it neither helps nor hurts the milestone; the
milestone does NOT enable R5 blob_fallback, so the 'R5 forced...' log lines emitted during
evaluate are harmless and ignored, as instructed.",
      "cap6->+R4 (the marginal R4 increment) is the weakest link of the chain: 1 loss / 3 ties
at n=13 and p=0.0215 -- still <0.05 but the only sub-12/0 sign-test; its lone loss is GX010137.
This is the additive R4 step, not the headline baseline->milestone comparison, which is far
stronger.",
      "over_seg_guard passes on the NET (rate-based) rule but strict_passes is False -- i.e. at
least one video's merge_rate is not strictly non-regressing; the net merge_rate still drops
sharply (0.651->0.161) and no_merge_rate_regress is True, consistent with the orchestrator's
claim of 'no honest over-seg regression'.",
      "n=13 is small; LOO at n=12 is the stress test requested and the verdict is unanimous, but
the bootstrap CI and sign-test both rest on a 12-13 video corpus -- robustness here means 'no
single clip flips the verdict', not large-sample certainty.",
      "Verified on origin/master f5f339a in the verify-n13 worktree using the public API
(backend.eval.rally_seg_eval / eval_stats / tolerance_metrics) against the absolute output/
golden dir; pure-logic path only (no GPU/video), which is the correct scope for this eval."
    ]
  },
  {
    "lens": "cap6->+R4 step (the inactivity-end trim): investigate the n=13 loss and whether +R4
still clears the net-over-seg promotion bar.",
    "verdict": "confirms",
    "key_numbers": "cap6->+R4 tolF1: mean +0.0397, CI [+0.0155,+0.0652] excludes 0, sign
9W/1L/3T, p=0.0215 -> clears bar (CI>0 AND p<0.05). The SINGLE loss = GX010137: tolF1
0.7273->0.6849 (delta -0.0423), but on that same clip strict f1 IMPROVES 0.8052->0.8219,
merge_rate(0.0)/split_rate(0.0294) UNCHANGED, num_pred 43->39 (closer to gt=34), and |dend|
IMPROVES 0.447->0.439 -> the dip is a tolerance-window accounting artifact, not an R4 failure.
Other 9 wins range +0.0038..+0.1189; 3 ties (mahadevpura_1=0, mahadevpura_2, testlarge_short).
Full milestone: baseline->milestone tolF1 +0.2222 [+0.1558,+0.2894] sign 12/0/1 p=0.0005;
cap12->cap6 +0.1099 [+0.0700,+0.1522] sign 12/0/1 p=0.0005. mean merge_rate 0.651->0.161.
baseline->milestone over_seg_guard PASSES (base_net 1.303->cand_net 0.429, net_delta -0.874, no
merge_rate regress). mean |dend| cap6 4.96 -> milestone 3.31; R4 worsens |dend| on ZERO clips
(worst = +0.011s, mahadevpura_singles). New clips: GX010137 baseline 0.448->0.685, GX010141
0.333->0.431 (both improve).",
    "discrepancies": [
      "cap6->+R4 CI lower bound: I got [+0.0155,+0.0652] vs reported [+0.0145,+0.0651] ? ~0.001
jitter, both exclude 0 (paired_delta_ci is bootstrap-resampled, so lower bound varies run-to-
run). Immaterial.",
      "baseline->milestone CI: I got [+0.1558,+0.2894] vs reported [+0.1522,+0.2859] ? ~0.003
bootstrap jitter, both well clear of 0. Immaterial.",
      "cap12->cap6 CI: I got [+0.0700,+0.1522] vs reported [+0.0703,+0.1526] ? ~0.0004 jitter.
Immaterial.",
      "All point estimates (mean deltas, sign W/L/T, p-values, merge_rate 0.651->0.161, the
single loser GX010137 at -0.0423, new-clip numbers) match exactly."
    ],
    "milestone_still_stands": true,
    "caveats": [
      "The one loss (GX010137, -0.0423) is the smallest non-zero delta in the loss column and is
noise-level / not a real R4 failure mode: on that clip strict f1 IMPROVES, |dend| IMPROVES, and
over-seg metrics are unchanged ? R4 simply trimmed 4 trailing predicted segments (43->39, toward
gt=34), which nudged the tolerance-match P/R balance down marginally.",
      "+R4 harms NO clip badly: zero clips show a large tolF1 drop beyond the one -0.0423, and
R4 never worsens |dend| by more than +0.011s on any clip; it improves |dend| on 11/13.",
      "The cap6->milestone over_seg_guard reports passes=False (net_delta -0.031, with
```

mahadevpura_2 merge_rate 0.475->0.525) ? but the PROMOTION bar is baseline->milestone, which PASSES cleanly (net_delta -0.874, no merge_rate regression). The cap6->milestone guard is an intermediate-step diagnostic, not the gate.",
      "All numbers computed at n=13 on the worktree at origin/master f5f339a, golden corpus from C:/Users/avidu/Projects/badminton-highlight-indexer/output (13 clips, all with gts + existing trajectory). CI bounds are bootstrap estimates and jitter ~0.001-0.003 between runs; this does not affect any excludes-zero / p<0.05 conclusion.",
      "This is a CPU-only pure-logic verification of the eval/windowing path; no GPU pipeline or real-video re-detection was run."
    ]
  },
  {
    "lens": "The 2 NEW clips (GX010137, GX010141): per-clip behavior under baseline vs the R1 milestone, and whether the new footage introduces a degenerate/blob failure mode or breaks the milestone story.",
    "verdict": "confirms",
    "key_numbers": "N=13 corpus confirmed. baseline->milestone tolF1 ?+0.2222 [+0.1522,+0.2859], sign 12/0/1, p=0.00049 (EXACT match). cap12->cap6 ?+0.1099 [+0.0703,+0.1526], 12/0/1, p=0.00049 (match). cap6->+R4 ?+0.0397 [+0.0145,+0.0651], 9/1/3, p=0.0215 (match). mean merge_rate 0.6513->0.1610 (match). over_seg_guard: passes=true, base_net=1.303->cand_net=0.429, net_delta=-0.874, no_merge_rate_regress=true (strict_passes=false but net PASS). NEW CLIPS: GX010137 baseline{num_gt34,num_pred33,tolF1 0.448,merge_rate0.176,split_rate0.0,dend1.218,maxwin~33s} -> milestone{num_pred39,tolF1 0.685,merge_rate0.0,split_rate0.029,dend0.439,maxwin~15s}. GX010141 baseline{num_gt29,num_pred19,tolF1 0.333,merge_rate0.655,split_rate0.0,dend5.431,maxwin~101s} -> milestone{num_pred22,tolF1 0.431,merge_rate0.448,split_rate0.0,dend0.676,maxwin~30s}. BOTH improve baseline->milestone (+0.237, +0.098).",
    "discrepancies": [
      "Orchestrator framed the new clips as straightforwardly 'both improve' ? TRUE for baseline->milestone, but GX010137 is precisely the ONE cap6->milestone LOSS the orchestrator flagged: 0.7273->0.6849 (?-0.0423). So the single regressing clip in the R4 step is one of the NEW clips, not legacy footage. Magnitude small (-0.042) and it still nets a large baseline win (+0.237); R4 trades a tiny tolF1 dip for a big end-precision gain on it (dend 1.218->0.439, merge_rate 0.176->0.0).",
      "over_seg_guard reports strict_passes=false (orchestrator said 'NET PASS' ? consistent, since net_cost_ok+no_merge_rate_regress carry the pass; strict fails only because R4 introduces new splits, e.g. GX010141 base 5 merges->6, and many clips gain splits). No magnitude conflict, just a nuance the headline omitted."
    ],
    "milestone_still_stands": true,
    "caveats": [
      "No new degenerate/blob failure mode from the new footage. Neither new clip collapses to a mahadevpura_1-style single-mega-window: GX010137 milestone = 39 windows (max ~15s), GX010141 = 22 windows (max ~30s). The genuine blob (mahadevpura_1: 1 window of 174s, tolF1=0.0) is legacy and unchanged; milestone does NOT enable R5 so the blob stays unsplit but contributes 0 either way.",
      "GX010141 remains the worst-behaved NEW clip post-milestone (merge_rate still 0.448, 6 merges, max window ~30s vs gt max 10.5s) ? it is partially-merged, not blob-like, and still improves (+0.098). It is a candidate future over-merge target but does NOT regress and does NOT break the guard.",
      "GX010137 is the lone cap6->milestone tolF1 loser; this is a within-step (R4) dip, NOT a baseline regression. It does not threaten the promotion bar, which is measured baseline->milestone (significant win) + over-seg guard net-pass ? both hold at n=13.",
      "The over-seg guard's new_splits list grows broadly under R4 (11 clips gain splits), which is why strict_passes=false; the net cost still drops 67% (1.30->0.43) and no clip's merge_rate regresses, so the honest-over-seg promotion bar is cleared. GX010141 appears in new_merges (5->6) but its merge_RATE drops (0.655->0.448), so no merge_rate regression.",
      "All independently recomputed via the public API (rse.evaluate / windows_for / eval_stats.paired_delta_ci / over_seg_guard) against the absolute output/ corpus on worktree HEAD f5f339a; numbers reproduce the orchestrator's to 4 dp."
    ]
  },
  {
    "lens": "Net over-segmentation guard at n=13: per-clip merge_rate/split_rate at baseline vs milestone, the categorical merge_rate-rise block condition, over_seg_guard verdict, and an

```
adversarial sweep over whether over-segmentation got honestly worse.",
    "verdict": "confirms",
    "key_numbers": "All 13 clips scored in both presets (none dropped;
set(baseline)==set(milestone)==13). over_seg_guard(base=baseline, cand=milestone): passes=True,
strict_passes=False, net_cost_ok=True (base_net 1.3026 -> cand_net 0.4289, net_delta -0.8737),
no_merge_rate_regress=True, merge_rate_regress=[] (EMPTY). ANY clip merge_rate increase
baseline->milestone: FALSE. mean merge_rate 0.6513 -> 0.1610; mean split_rate 0.0000 -> 0.1069
(baseline split_count all-zero by construction). tolF1 paired deltas (n=13): baseline->milestone
?+0.2222 [+0.1558,+0.2894] sign 12/0/1 p=0.0005; cap12->cap6 ?+0.1099 [+0.0700,+0.1522] sign
12/0/1 p=0.0005; cap6->milestone ?+0.0397 [+0.0155,+0.0652] sign 9/1/3 p=0.0215. The ONE
cap6->milestone loss = GX010137 (0.7273 -> 0.6849, ?-0.0423). New clips: GX010137 tolF1
0.4478->0.6849, GX010141 0.3333->0.4314 (both improve). Adversarial net cost: down under 2:1
(1.3026->0.4289), 1:1 (0.6513->0.2679), 1:2 (0.6513->0.3748); rises ONLY under split-punitive
1:5 (0.6513->0.6953). Zero clips regress on BOTH merge_rate AND split_rate.",
    "discrepancies": [
      "baseline->milestone tolF1 CI: I got [+0.1558,+0.2894], orchestrator reported
[+0.1522,+0.2859] ? bootstrap-seed/sampling jitter, ~0.0035 on each bound; mean ?+0.2222, sign
12/0/1, p=0.0005 match exactly. Inconsequential.",
      "cap12->cap6 tolF1 CI: I got [+0.0700,+0.1522], orchestrator [+0.0703,+0.1526] ? same
~0.0004 bootstrap jitter; mean ?+0.1099, sign 12/0/1, p=0.0005 match.",
      "cap6->milestone tolF1 CI: I got [+0.0155,+0.0652], orchestrator [+0.0145,+0.0651]; mean
?+0.0397, sign 9/1/3, p=0.0215, and the single loss=GX010137 all match.",
      "LENS-FRAMING (not a number discrepancy): the lens asks me to confirm strict BLOCKS
'purely because of split-count rises'. It does NOT ? strict trips on BOTH new_splits (11/13
clips, 0->N, structural) AND new_merges (4 clips: GX010141 5->6, kushagra 2->3, mahadevpura_1
1->2, mahadevpura_2 3->9). Crucially, all 4 merge_COUNT rises have merge_RATE FALLING
(0.655->0.448, 1.000->0.188, 1.000->0.800, 0.975->0.525) ? the mega-window->bounded artifact
(one window hiding N rallies becomes several each gluing 2 neighbours: higher count, lower
fraction-hidden). So strict blocks on a count-monotonicity artifact on BOTH axes, not just
split; no real over-merge regression exists. The net-guard's merge_RATE-based rule correctly
passes."
    ],
    "milestone_still_stands": true,
    "caveats": [
      "mean split_rate rises 0.0000->0.1069 and mean num_pred inflates 23.77->57.69 ? but this
is the intended structural mega->bounded artifact (a split still FINDS the rally; only a merge
hides it), not new over-merging. No clip regresses on both axes; zero merge_rate regressions.",
      "The promote verdict survives every defensible over-seg weighting (merge:split = 2:1
default, 1:1 equal, 1:2 split-heavy all show net cost FALLING). It flips to a net-cost RISE only
under an indefensible split-punitive 1:5 weight, which would rank a fragmented-but-found rally
2.5x worse than a hidden rally ? an inversion of the recall-meaningful priority. So 'over-
segmentation got worse' is reachable only by an absurd weighting; under any honest reading it
improved.",
      "tolF1 CIs are bootstrap (n_boot=2000, seed-dependent) so bounds wobble ~0.001-0.004 run-
to-run; means, sign tests, and p-values are exact and match the orchestrator. scipy/numpy
present here so the Hungarian 1:1 matcher (not the greedy fallback) was used.",
      "rse.evaluate emitted the harmless R5 blob-fallback log lines (filtered out); milestone
does NOT enable blob_fallback, so they are irrelevant to this verdict.",
      "This verifies the net-over-seg guard + tolF1 significance only; it does not re-run the
full test suite, lint, type checks, or coverage (out of scope for this lens) and the CV pipeline
cannot run in this container."
    ]
  }
]

Render the FINAL verdict on
whether config-flip PR #204 (the R1 milestone default flip) still stands at n=13. Weigh
refutations and
caveats honestly ? a single credible refutation downgrades the verdict. Distinguish "the
milestone win
stands" (the headline) from any softening (e.g. R4's marginal step). Be specific and honest.
```

**2. assistant (2026-06-18T05:35:30.440Z)**

I'll independently verify this claim. Let me start by examining the worktree and reproducing the

numbers.

**3. user (2026-06-18T05:35:32.222Z)**

f5f339a fix(eval): fusion loaders skip windowing-only files on a mixed golden corpus (#210)

**4. user (2026-06-18T05:35:33.296Z)**

C:/Users/avidu/Projects/badminton-highlight-indexer/output/human_lovo_manifest.json
__init__.py
__pycache__
ablation.py
annotations.py
audio
batch.py
calibrate_local.py
calibrate_wasb.py
calibration.py
classifier.py
distill_local.py
eval_partial_labels.py
eval_stats.py
experiment.py
export_wasb_segments.py
fusion_audio.py
fusion_compare.py
fusion_golden.py
gemini_refine.py
golden_fixtures.py
gt_loader.py
harness.py
metrics.py
per_user_eval.py
per_user_gate.py
per_user_regression.py
promotion.py
rally_seg_eval.py
rally_seq_proto.py
regression.py

**5. assistant (2026-06-18T05:35:39.535Z)**

The worktree is at the right commit (f5f339a includes #210 on top of #204's f5f5e52). Let me write my own independent reproduction script.

**6. user (2026-06-18T05:35:41.220Z)**

analyze_motion.py
clean_state.py
diagnose_rally.py
gemini_video_debugger.py
lens_cap6_r4.py
lens_gx137.py
lens_loo.py
lens_n13_overseg.py
lens_new_clips.py
lens_verify_n13.py

**7. assistant (2026-06-18T05:35:44.135Z)**

The lens scripts already exist. Let me write my own fresh independent script rather than reuse theirs, to avoid trusting their code.

**8. assistant (2026-06-18T05:35:57.262Z)**

```
import os
from dataclasses import replace
from backend.eval import rally_seg_eval as rse, eval_stats
from backend.eval.windowing import SERVED
from backend.eval.tolerance_metrics import over_seg_guard

OUT = "C:/Users/avidu/Projects/badminton-highlight-indexer/output"

golden = [g for g in rse.load_golden(os.path.join(OUT, "human_lovo_manifest.json"), OUT)
          if g["gts"] and os.path.exists(g["trajectory"])]
print("N golden =", len(golden))
print("names =", sorted(g["name"] for g in golden))

baseline = SERVED
cap12 = replace(SERVED, max_window_duration=12.0)
cap6 = replace(SERVED, max_window_duration=6.0)
milestone = replace(SERVED, max_window_duration=6.0, inactivity_end=1.5,
                    inactivity_rally_thresh=0.20, inactivity_min_density=0.30)


def rows(p):
    return {r["name"]: r for r in rse.evaluate(golden, p)["rows"]}


base_r = rows(baseline)
cap12_r = rows(cap12)
cap6_r = rows(cap6)
mile_r = rows(milestone)

names = sorted(base_r.keys())
print("\nrow names count:", len(names))


def delta(a, b, k):
    return eval_stats.paired_delta_ci([a[n][k] for n in names], [b[n][k] for n in names])


print("\n=== tolF1 paired deltas ===")
for label, a, b in [("baseline->milestone", base_r, mile_r),
                    ("cap12->cap6", cap12_r, cap6_r),
                    ("cap6->+R4(milestone)", cap6_r, mile_r)]:
    d = delta(a, b, "tol_f1")
    st = d["sign_test"]
    print(f"{label}: mean={d['mean_delta']:+.4f} CI[{d['ci_low']:+.4f},{d['ci_high']:+.4f}] "
          f"excl0={d['ci_excludes_zero']} sign {st['wins']}/{st['losses']}/{st['ties']}
p={st['p_value']:.4f}")
```

## find the cap6->milestone loss(es)

```
print("\n=== cap6->milestone per-clip tolF1 (losses/ties flagged) ===")
for n in names:
    d = mile_r[n]["tol_f1"] - cap6_r[n]["tol_f1"]
    flag = "LOSS" if d < -1e-9 else ("TIE" if abs(d) <= 1e-9 else "")
    if flag:
        print(f"  {n}: {cap6_r[n]['tol_f1']:.4f} -> {mile_r[n]['tol_f1']:.4f}  d={d:+.4f}
{flag}")

print("\n=== merge_rate baseline->milestone ===")
mr_base = sum(base_r[n]["merge_rate"] for n in names) / len(names)
mr_mile = sum(mile_r[n]["merge_rate"] for n in names) / len(names)
print(f"mean merge_rate baseline={mr_base:.4f} milestone={mr_mile:.4f}")

print("\n=== over_seg_guard baseline->milestone ===")
g = over_seg_guard(list(base_r.values()), list(mile_r.values()))
print(g)
```

```
print("\n=== new clips ===")
for n in ["GX010137", "GX010141"]:
    if n in names:
        print(f"  {n}: baseline tolF1 {base_r[n]['tol_f1']:.4f} -> milestone
{mile_r[n]['tol_f1']:.4f}")

print("\n=== mean tolF1 per preset ===")
for label, r in [("baseline", base_r), ("cap12", cap12_r), ("cap6", cap6_r), ("milestone",
mile_r)]:
    m = sum(r[n]["tol_f1"] for n in names) / len(names)
    print(f"  {label}: {m:.4f}")
```

**9. user (2026-06-18T05:35:57.285Z)**

```
File created successfully at: C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/verify-n13/scratch/lens_judge.py (file state is current in your
context ? no need to Read it back)
```

**10. user (2026-06-18T05:36:10.938Z)**

```
N golden = 13
names = ['Badminton_BXH_2', 'GX010128', 'GX010137', 'GX010141', 'GX030094',
'adarsh_avi_singles', 'gbaaddy', 'kushagra_singles', 'largetest_doubles', 'mahadevpura_1',
'mahadevpura_2', 'mahadevpura_singles', 'testlarge_short']

row names count: 13

=== tolF1 paired deltas ===
baseline->milestone: mean=+0.2222 CI[+0.1558,+0.2894] excl0=True sign 12.0/0.0/1.0 p=0.0005
cap12->cap6: mean=+0.1099 CI[+0.0700,+0.1522] excl0=True sign 12.0/0.0/1.0 p=0.0005
cap6->+R4(milestone): mean=+0.0397 CI[+0.0155,+0.0652] excl0=True sign 9.0/1.0/3.0 p=0.0215

=== cap6->milestone per-clip tolF1 (losses/ties flagged) ===
  GX010137: 0.7273 -> 0.6849  d=-0.0423  LOSS
  mahadevpura_1: 0.0000 -> 0.0000  d=+0.0000  TIE
  mahadevpura_2: 0.1220 -> 0.1220  d=+0.0000  TIE
  testlarge_short: 0.2000 -> 0.2000  d=+0.0000  TIE

=== merge_rate baseline->milestone ===
mean merge_rate baseline=0.6513 milestone=0.1610

=== over_seg_guard baseline->milestone ===
{'passes': True, 'strict_passes': False, 'net_cost_ok': True, 'no_merge_rate_regress': True,
'base_net': 1.3025976530825853, 'cand_net': 0.4289172588246174, 'net_delta': -0.873680394257968,
'n': 13, 'use_rates': True, 'weight_merge': 2.0, 'weight_split': 1.0, 'new_merges': [{'name':
'kushagra_singles', 'base': 2.0, 'cand': 3.0}, {'name': 'mahadevpura_2', 'base': 3.0, 'cand':
9.0}, {'name': 'mahadevpura_1', 'base': 1.0, 'cand': 2.0}, {'name': 'GX010141', 'base': 5.0,
'cand': 6.0}], 'new_splits': [{'name': 'adarsh_avi_singles', 'base': 0.0, 'cand': 13.0},
{'name': 'kushagra_singles', 'base': 0.0, 'cand': 6.0}, {'name': 'largetest_doubles', 'base':
0.0, 'cand': 5.0}, {'name': 'gbaaddy', 'base': 0.0, 'cand': 8.0}, {'name': 'testlarge_short',
'base': 0.0, 'cand': 1.0}, {'name': 'mahadevpura_singles', 'base': 0.0, 'cand': 7.0}, {'name':
'mahadevpura_2', 'base': 0.0, 'cand': 4.0}, {'name': 'GX010128', 'base': 0.0, 'cand': 2.0},
{'name': 'GX030094', 'base': 0.0, 'cand': 6.0}, {'name': 'Badminton_BXH_2', 'base': 0.0, 'cand':
4.0}, {'name': 'GX010137', 'base': 0.0, 'cand': 1.0}], 'merge_rate_regress': []}

=== new clips ===
  GX010137: baseline tolF1 0.4478 -> milestone 0.6849
  GX010141: baseline tolF1 0.3333 -> milestone 0.4314

=== mean tolF1 per preset ===
  baseline: 0.1372
  cap12: 0.2099
  cap6: 0.3198
  milestone: 0.3594
```

**11. assistant (2026-06-18T05:36:13.983Z)**

All headline numbers reproduce exactly. Now let me independently verify the LOO robustness and probe the one identified loss (GX010137) more deeply, plus run an adversarial check on the over-seg guard weighting.

**12. assistant (2026-06-18T05:36:29.856Z)**

```
import os
from dataclasses import replace
from backend.eval import rally_seg_eval as rse, eval_stats
from backend.eval.windowing import SERVED
from backend.eval.tolerance_metrics import over_seg_guard

OUT = "C:/Users/avidu/Projects/badminton-highlight-indexer/output"

golden = [g for g in rse.load_golden(os.path.join(OUT, "human_lovo_manifest.json"), OUT)
          if g["gts"] and os.path.exists(g["trajectory"])]

baseline = SERVED
cap6 = replace(SERVED, max_window_duration=6.0)
milestone = replace(SERVED, max_window_duration=6.0, inactivity_end=1.5,
                    inactivity_rally_thresh=0.20, inactivity_min_density=0.30)


def rows(p, gs):
    return {r["name"]: r for r in rse.evaluate(gs, p)["rows"]}


base_r = rows(baseline, golden)
mile_r = rows(milestone, golden)
cap6_r = rows(cap6, golden)
names = sorted(base_r.keys())
```

## === LOO robustness on baseline->milestone tolF1 ===

```
print("=== LOO baseline->milestone tolF1 (drop each clip, n=12) ===")
worst = None
all_pass = True
for drop in names:
    keep = [n for n in names if n != drop]
    d = eval_stats.paired_delta_ci([base_r[n]["tol_f1"] for n in keep],
                                   [mile_r[n]["tol_f1"] for n in keep])
    st = d["sign_test"]
    ok = d["ci_excludes_zero"] and st["p_value"] < 0.05
    all_pass = all_pass and ok
    print(f"  drop {drop:22s}: mean={d['mean_delta']:+.4f}
CI[{d['ci_low']:+.4f},{d['ci_high']:+.4f}] "
          f"sign {st['wins']:.0f}/{st['losses']:.0f}/{st['ties']:.0f} p={st['p_value']:.4f}
{'OK' if ok else 'FAIL'}")
    if worst is None or d["mean_delta"] < worst[1]:
        worst = (drop, d["mean_delta"], d["ci_low"], st["p_value"])
print(f"ALL LOO folds clear CI-excl-0 AND p<0.05: {all_pass}")
print(f"weakest fold: drop {worst[0]} mean={worst[1]:+.4f} ci_low={worst[2]:+.4f}
p={worst[3]:.4f}")
```

## === Deep dive on GX010137 (the cap6->milestone loss) ===

```
print("\n=== GX010137 deep dive (cap6 vs milestone) ===")
for k in ["tol_f1", "f1", "merge_rate", "split_rate", "merge_count", "split_count",
          "num_pred", "num_gt", "dend_abs_median"]:
    cv = cap6_r["GX010137"].get(k)
    mv = mile_r["GX010137"].get(k)
    print(f"  {k:18s}: cap6={cv}  milestone={mv}")
```

## === Adversarial over_seg_guard weighting sweep ===

```
print("\n=== over_seg_guard weighting sweep (baseline->milestone) ===")
import inspect
sig = inspect.signature(over_seg_guard)
print("  over_seg_guard params:", list(sig.parameters.keys()))
for wm, ws in [(2.0, 1.0), (1.0, 1.0), (1.0, 2.0), (1.0, 5.0)]:
    try:
        g = over_seg_guard(list(base_r.values()), list(mile_r.values()),
                           weight_merge=wm, weight_split=ws)
        print(f"  merge:split={wm}:{ws} base_net={g['base_net']:.4f}
cand_net={g['cand_net']:.4f} "
              f"net_delta={g['net_delta']:+.4f} passes={g['passes']}")
    except TypeError as e:
        print(f"  weighting not parametrizable via kwargs: {e}")
        break
```

## === merge_rate per-clip: any regression baseline->milestone? ===

```
print("\n=== per-clip merge_rate baseline->milestone (any rise?) ===")
any_rise = False
for n in names:
    b = base_r[n]["merge_rate"]
    m = mile_r[n]["merge_rate"]
    if m > b + 1e-9:
        any_rise = True
        print(f"  RISE {n}: {b:.4f} -> {m:.4f}")
print(f"any merge_rate rise: {any_rise}")
```

**13. user (2026-06-18T05:36:29.872Z)**

```
File created successfully at: C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/verify-n13/scratch/lens_judge2.py (file state is current in your
context ? no need to Read it back)
```

**14. user (2026-06-18T05:36:41.489Z)**

```
=== LOO baseline->milestone tolF1 (drop each clip, n=12) ===
  drop Badminton_BXH_2       : mean=+0.2134 CI[+0.1447,+0.2872] sign 11/0/1 p=0.0010 OK
  drop GX010128              : mean=+0.2019 CI[+0.1409,+0.2608] sign 11/0/1 p=0.0010 OK
  drop GX010137              : mean=+0.2210 CI[+0.1516,+0.2974] sign 11/0/1 p=0.0010 OK
  drop GX010141              : mean=+0.2325 CI[+0.1630,+0.3037] sign 11/0/1 p=0.0010 OK
  drop GX030094              : mean=+0.2175 CI[+0.1480,+0.2933] sign 11/0/1 p=0.0010 OK
  drop adarsh_avi_singles    : mean=+0.2267 CI[+0.1556,+0.3012] sign 11/0/1 p=0.0010 OK
  drop gbaaddy               : mean=+0.2176 CI[+0.1491,+0.2919] sign 11/0/1 p=0.0010 OK
  drop kushagra_singles      : mean=+0.2342 CI[+0.1686,+0.2993] sign 11/0/1 p=0.0010 OK
  drop largetest_doubles     : mean=+0.2169 CI[+0.1470,+0.2886] sign 11/0/1 p=0.0010 OK
  drop mahadevpura_1         : mean=+0.2407 CI[+0.1839,+0.3037] sign 12/0/0 p=0.0005 OK
  drop mahadevpura_2         : mean=+0.2306 CI[+0.1636,+0.2993] sign 11/0/1 p=0.0010 OK
  drop mahadevpura_singles   : mean=+0.2117 CI[+0.1447,+0.2828] sign 11/0/1 p=0.0010 OK
  drop testlarge_short       : mean=+0.2241 CI[+0.1526,+0.2980] sign 11/0/1 p=0.0010 OK
ALL LOO folds clear CI-excl-0 AND p<0.05: True
weakest fold: drop GX010128 mean=+0.2019 ci_low=+0.1409 p=0.0010

=== GX010137 deep dive (cap6 vs milestone) ===
  tol_f1            : cap6=0.7272727272727273  milestone=0.6849315068493151
  f1                : cap6=0.8051948051948051  milestone=0.8219178082191781
  merge_rate        : cap6=0.0  milestone=0.0
  split_rate        : cap6=0.029411764705882353  milestone=0.029411764705882353
  merge_count       : cap6=0  milestone=0
  split_count       : cap6=1  milestone=1
  num_pred          : cap6=43  milestone=39
  num_gt            : cap6=34  milestone=34
  dend_abs_median   : cap6=0.4470932599265893  milestone=0.43861027694362065
```

```
=== over_seg_guard weighting sweep (baseline->milestone) ===
  over_seg_guard params: ['base', 'cand', 'weight_merge', 'weight_split', 'use_rates', 'eps']
  merge:split=2.0:1.0 base_net=1.3026 cand_net=0.4289 net_delta=-0.8737 passes=True
  merge:split=1.0:1.0 base_net=0.6513 cand_net=0.2679 net_delta=-0.3834 passes=True
  merge:split=1.0:2.0 base_net=0.6513 cand_net=0.3748 net_delta=-0.2765 passes=True
  merge:split=1.0:5.0 base_net=0.6513 cand_net=0.6953 net_delta=+0.0440 passes=False

=== per-clip merge_rate baseline->milestone (any rise?) ===
any merge_rate rise: False
```

**15. assistant (2026-06-18T05:36:46.110Z)**

Every number reproduces exactly. Let me verify a couple of the claims made in the caveats: (1) the over_seg_guard `passes` is driven by `net_cost_ok AND no_merge_rate_regress` (not strict), and (2) GX010137's strict f1 improves and dend improves under R4 ? both confirmed above (f1 0.805?0.822, dend 0.447?0.439, merge_rate unchanged at 0.0). Let me confirm the guard's pass logic in source and check bootstrap-seed sensitivity on the weakest fold.

**16. user (2026-06-18T05:36:49.413Z)**

```
    """**Net** over-segmentation promotion guard (the R1 refinement of the strict per-video
rule).
    **Why the strict rule needed refining.** R2 §2.3.3's guard blocks promotion if ``cand``
raises
    So the strict count rule blocks a config that strictly *reduces* the fraction of rallies
lost to
    over-merge. (Measured: the cap6+R4 milestone trips strict on 10/11 videos yet cuts mean
    **The net rule (default verdict ``passes``):** score over-seg as ONE weighted cost per video
and
    **Revert (one line):** read ``strict_passes`` instead of ``passes`` to fall back to the
original
    strict per-video count rule. Both verdicts are always reported, so the choice is auditable.
    new_merges: List[Dict[str, Any]] = []           # raw COUNT increases (what strict trips on)
    new_splits: List[Dict[str, Any]] = []           # raw COUNT increases (what strict trips on)
    strict_passes = (not new_merges) and (not new_splits)
    net_cost_ok = net_delta <= eps
    no_merge_rate_regress = not merge_rate_regress
    passes = bool(net_cost_ok and no_merge_rate_regress)
        "passes": passes,
        "strict_passes": strict_passes,
        "net_cost_ok": net_cost_ok,
        "no_merge_rate_regress": no_merge_rate_regress,
```

**17. assistant (2026-06-18T05:36:54.437Z)**

Confirmed: `passes = net_cost_ok AND no_merge_rate_regress`. This is the documented R1 net-rule design ? the promotion gate is the net rate-based rule, not the strict count rule. The docstring explicitly explains why strict needed refining (it blocks a config that strictly *reduces* over-merge), and both verdicts are always reported for auditability. This is honest, not a goalpost-move.

Let me do one final adversarial check: confirm bootstrap-seed sensitivity doesn't flip the weakest fold, and verify the cap6?milestone step's p=0.0215 isn't a borderline artifact by checking the sign-test exactly.

**18. user (2026-06-18T05:37:07.301Z)**

```
paired_delta_ci params: ['a_values', 'b_values', 'confidence', 'n_boot', 'seed', 'eps']
  seed=0: CI[+0.1409,+0.2608] excl0=True
  seed=1: CI[+0.1394,+0.2589] excl0=True
  seed=2: CI[+0.1444,+0.2590] excl0=True
  seed=7: CI[+0.1406,+0.2579] excl0=True
  seed=42: CI[+0.1431,+0.2598] excl0=True
```

**19. assistant (2026-06-18T05:37:11.888Z)**

The weakest LOO fold's CI robustly excludes zero across all bootstrap seeds (ci_low never drops below +0.139). The CI jitter noted by the lenses (~0.001?0.004 vs orchestrator) is explained: bootstrap resampling with different seeds, immaterial to every excludes-zero / p<0.05 conclusion.

I have independently confirmed every load-bearing number. Let me render the final verdict.

**20. user (2026-06-18T05:37:45.228Z)**

Structured output provided successfully

**21. assistant (2026-06-18T05:37:53.824Z)**

Verdict rendered: **STANDS**.

I independently reproduced every load-bearing number with my own fresh script (`scratch/lens_judge.py`, `scratch/lens_judge2.py`) on worktree HEAD f5f339a against the absolute `output/` golden dir at n=13. The orchestrator's numbers and all five lenses confirm; I found no credible refutation, only immaterial bootstrap-CI jitter (traced to seed-dependent resampling ? the weakest LOO fold's ci_low stays in [+0.139,+0.144] across seeds 0/1/2/7/42).

Key independent confirmations:
- baseline?milestone tolF1 ?+0.2222, sign 12/0/1, p=0.0005; over_seg_guard NET PASS (net_delta ?0.8737, zero merge_rate regressions).
- All 13 LOO folds clear CI-excl-0 AND p<0.05; weakest (drop GX010128) still ?+0.2019, p=0.0010 ? no single clip drives the win.
- The one cap6?+R4 loss (GX010137, ?0.0423) is a tolerance-accounting artifact: on that clip strict f1 *improves* 0.805?0.822, |dend| improves, merge_rate unchanged.
- `passes = net_cost_ok AND no_merge_rate_regress` confirmed in source; strict_passes=False is the documented R1 design, not a moved goalpost.

The headline win stands solidly; the only softening is the marginal (but still significant) R4 step.